

# Programming

optimisation and operations research algorithms with Julia

## for Business Tasks

Prof. Dr. Xavier Gandibleux

Université de Nantes  
Département Informatique – Faculté des Sciences et Techniques  
France

Lesson 1 – May-June 2022



version 1.7.x and later

# Why Julia?

Discovered in 2016 (v0.4.5), it became my favourite language:

## 1. Runs like C, walks like Python, talks like Lisp

- Same speed as C or FORTRAN code
- Development dynamic like writing python code
- Less object-oriented more function-oriented (like Lisp)

⇒ High-level and high-performance!

⇒ Extremely good ratio between coding time and running time.

⇒ Familiar for the users of Matlab, Fortran, Python, C/C++, etc.

## 2. Additional value

- Open source: MIT license, source code available on GitHub
- Multi-platform (x86, ARM, Apple M-series -experimental-, etc.)
- Cross-language support: C, FORTRAN, python, R ... from Julia
- Rich set of packages: Optimisation, Plotting, Visualisation, Statistics, Data science, GPU, etc.

⇒ Ideal for the development of efficient open-source app

# Why Julia?

Discovered in 2016 (v0.4.5), it became my favourite language:

## 1. Runs like C, walks like Python, talks like Lisp

- Same speed as C or FORTRAN code
- Development dynamic like writing python code
- Less object-oriented more function-oriented (like Lisp)

⇒ High-level and high-performance!

⇒ Extremely good ratio between coding time and running time.

⇒ Familiar for the users of Matlab, Fortran, Python, C/C++, etc.

## 2. Additional value

- Open source: MIT license, source code available on GitHub
- Multi-platform (x86, ARM, Apple M-series -experimental-, etc.)
- Cross-language support: C, FORTRAN, python, R ... from Julia
- Rich set of packages: Optimisation, Plotting, Visualisation, Statistics, Data science, GPU, etc.

⇒ Ideal for the development of efficient open-source app

# Why Julia?

Discovered in 2016 (v0.4.5), it became my favourite language:

## 1. Runs like C, walks like Python, talks like Lisp

- Same speed as C or FORTRAN code
- Development dynamic like writing python code
- Less object-oriented more function-oriented (like Lisp)

⇒ High-level and high-performance!

⇒ Extremely good ratio between coding time and running time.

⇒ Familiar for the users of Matlab, Fortran, Python, C/C++, etc.

## 2. Additional value

- Open source: MIT license, source code available on GitHub
- Multi-platform (x86, ARM, Apple M-series -experimental-, etc.)
- Cross-language support: C, FORTRAN, python, R ... from Julia
- Rich set of packages: Optimisation, Plotting, Visualisation, Statistics, Data science, GPU, etc.

⇒ Ideal for the development of efficient open-source app

# Why Julia?

Discovered in 2016 (v0.4.5), it became my favourite language:

## 1. Runs like C, walks like Python, talks like Lisp

- Same speed as C or FORTRAN code
- Development dynamic like writing python code
- Less object-oriented more function-oriented (like Lisp)

⇒ High-level and high-performance!

⇒ Extremely good ratio between coding time and running time.

⇒ Familiar for the users of Matlab, Fortran, Python, C/C++, etc.

## 2. Additional value

- Open source: MIT license, source code available on GitHub
- Multi-platform (x86, ARM, Apple M-series -experimental-, etc.)
- Cross-language support: C, FORTRAN, python, R ... from Julia
- Rich set of packages: Optimisation, Plotting, Visualisation, Statistics, Data science, GPU, etc.

⇒ Ideal for the development of efficient open-source app



## A high-level, high-performance programming language for scientific computing <https://julialang.org/>

- ▶ Timeline:  
2009...2012 @ MIT, USA  
version 1.0.0 in 2018; version 1.7.2 (February 6, 2022)
- ▶ Communities:  
Over 7,400 Julia packages <https://juliahub.com/ui/Packages>  
Optimization <https://jump.dev/>
- ▶ JuliaCon...ferences:  
every year since 2014 <https://juliacon.org/>  
27th to 29th of July, 2022: free, online and everywhere
- ▶ Structures:  
JuliaComputing <http://juliacomputing.com>  
JuliaLab <http://julia.mit.edu>

Jeff Bezanson, Alan Edelman, Stefan Karpinski and Viral B. Shah: Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59: 65–98. 2017.



## A high-level, high-performance programming language for scientific computing <https://julialang.org/>

- ▶ Timeline:  
2009...2012 @ MIT, USA  
version 1.0.0 in 2018; version 1.7.2 (February 6, 2022)
- ▶ Communities:  
Over 7,400 Julia packages <https://juliahub.com/ui/Packages>  
Optimization <https://jump.dev/>
- ▶ JuliaCon...ferences:  
every year since 2014 <https://juliacon.org/>  
27th to 29th of July, 2022: free, online and everywhere
- ▶ Structures:  
JuliaComputing <http://juliacomputing.com>  
JuliaLab <http://julia.mit.edu>

Jeff Bezanson, Alan Edelman, Stefan Karpinski and Viral B. Shah: Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59: 65–98. 2017.



## A high-level, high-performance programming language for scientific computing <https://julialang.org/>

- ▶ Timeline:  
2009...2012 @ MIT, USA  
version 1.0.0 in 2018; version 1.7.2 (February 6, 2022)
- ▶ Communities:  
Over 7,400 Julia packages <https://juliahub.com/ui/Packages>  
Optimization <https://jump.dev/>
- ▶ JuliaCon...ferences:  
every year since 2014 <https://juliacon.org/>  
27th to 29th of July, 2022: free, online and everywhere
- ▶ Structures:  
JuliaComputing <http://juliacomputing.com>  
JuliaLab <http://julia.mit.edu>

Jeff Bezanson, Alan Edelman, Stefan Karpinski and Viral B. Shah: Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59: 65–98. 2017.



## A high-level, high-performance programming language for scientific computing <https://julialang.org/>

- ▶ Timeline:  
2009...2012 @ MIT, USA  
version 1.0.0 in 2018; version 1.7.2 (February 6, 2022)
- ▶ Communities:  
Over 7,400 Julia packages <https://juliahub.com/ui/Packages>  
Optimization <https://jump.dev/>
- ▶ JuliaCon...ferences:  
every year since 2014 <https://juliacon.org/>  
27th to 29th of July, 2022: free, online and everywhere
- ▶ Structures:  
JuliaComputing <http://juliacomputing.com>  
JuliaLab <http://julia.mit.edu>

Jeff Bezanson, Alan Edelman, Stefan Karpinski and Viral B. Shah: Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59: 65–98. 2017.

# Key concepts

- ▶ Julia has an LLVM-based Just-in-time compilation (JIT)
- ▶ Compilation down to native code (running on CPU and GPU, ...)
- ▶ Julia is dynamically typed, provides multiple dispatch
- ▶ Julia has many built-in mathematical functions
- ▶ A built-in package manager
- ▶ Designed for parallelism and distributed computation

# Resources

Some URL to know:

- ▶ Official website

<https://julialang.org/>

- ▶ Official documentation (online and pdf)

<https://docs.julialang.org/>

- ▶ Julia channel on *youtube*

[https://www.youtube.com/channel/UC9IuUwwE2xdjQUT\\_LMLONoA/featured](https://www.youtube.com/channel/UC9IuUwwE2xdjQUT_LMLONoA/featured)

- ▶ Questions and discussion on *discourse*

<https://discourse.julialang.org/>

- ▶ Book online « Think Julia: How to Think Like a Computer Scientist »

<https://benlauwens.github.io/ThinkJulia.jl/latest/book.html>

- ▶ Julia (v1.6.3) on *replit*

<https://replit.com/languages/julia>

# Resources

Some URL to know:

- ▶ Official website  
<https://julialang.org/>
- ▶ Official documentation (online and pdf)  
<https://docs.julialang.org/>
- ▶ Julia channel on *youtube*  
[https://www.youtube.com/channel/UC9IuUwwE2xdjQUT\\_LMLONoA/featured](https://www.youtube.com/channel/UC9IuUwwE2xdjQUT_LMLONoA/featured)
- ▶ Questions and discussion on *discourse*  
<https://discourse.julialang.org/>
- ▶ Book online « Think Julia: How to Think Like a Computer Scientist »  
<https://benlauwens.github.io/ThinkJulia.jl/latest/book.html>
- ▶ Julia (v1.6.3) on *replit*  
<https://replit.com/languages/julia>



# Getting started

## Software environment



# Software environment: check list

## Base:

- ▶ Operating system: **MacOS**, Windows, Linux (ubuntu), etc.
- ▶ Browser: **Firefox**, Safari, Chrome, etc.

## Apps to install on your computer:

- ▶ Language: **Julia 1.7.2**
- ▶ Editor: **Visual Studio Code (VSCode)**, Emacs, etc.

## Apps to install for optimisation needs:

- ▶ Open-source MILP solver: **GLPK**, COIN-OR, etc.
- ▶ Commercial MILP solver: **Gurobi**, Cplex, etc.

## Apps used (installation not mandatory but recommended):

- ▶ interacting with repository: **GitHub Desktop**

# Software environment: check list

## Base:

- ▶ Operating system: **MacOS**, Windows, Linux (ubuntu), etc.
- ▶ Browser: **Firefox**, Safari, Chrome, etc.

## Apps to install on your computer:

- ▶ Language: **Julia 1.7.2**
- ▶ Editor: **Visual Studio Code (VSCode)**, Emacs, etc.

## Apps to install for optimisation needs:

- ▶ Open-source MILP solver: **GLPK**, COIN-OR, etc.
- ▶ Commercial MILP solver: **Gurobi**, Cplex, etc.

## Apps used (installation not mandatory but recommended):

- ▶ interacting with repository: **GitHub Desktop**

# Software environment: check list

## Base:

- ▶ Operating system: **MacOS**, Windows, Linux (ubuntu), etc.
- ▶ Browser: **Firefox**, Safari, Chrome, etc.

## Apps to install on your computer:

- ▶ Language: **Julia 1.7.2**
- ▶ Editor: **Visual Studio Code (VSCode)**, Emacs, etc.

## Apps to install for optimisation needs:

- ▶ Open-source MILP solver: **GLPK**, COIN-OR, etc.
- ▶ Commercial MILP solver: **Gurobi**, Cplex, etc.

## Apps used (installation not mandatory but recommended):

- ▶ interacting with repository: **GitHub Desktop**

# Software environment: check list

## Base:

- ▶ Operating system: **MacOS**, Windows, Linux (ubuntu), etc.
- ▶ Browser: **Firefox**, Safari, Chrome, etc.

## Apps to install on your computer:

- ▶ Language: **Julia 1.7.2**
- ▶ Editor: **Visual Studio Code (VSCode)**, Emacs, etc.

## Apps to install for optimisation needs:

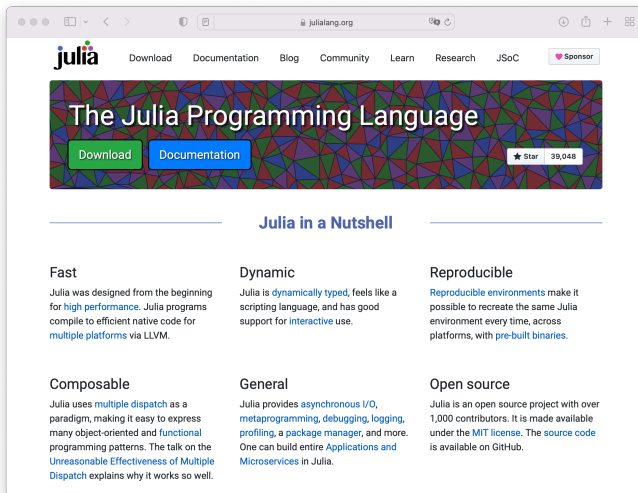
- ▶ Open-source MILP solver: **GLPK**, COIN-OR, etc.
- ▶ Commercial MILP solver: **Gurobi**, Cplex, etc.

## Apps used (installation not mandatory but recommended):

- ▶ interacting with repository: **GitHub Desktop**

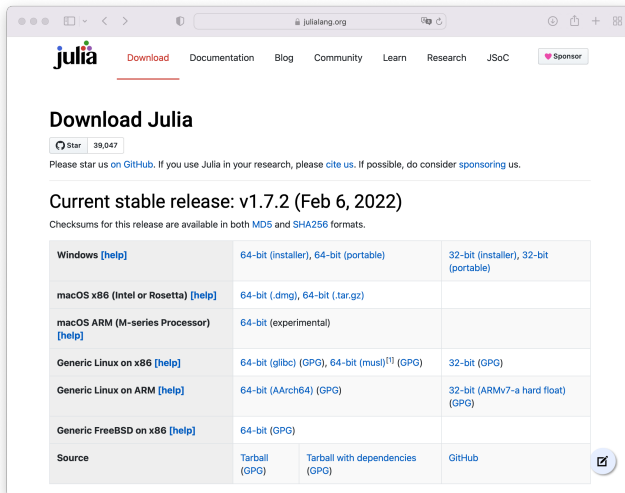
# Language: Julia 1.7.2

`https://julialang.org/`



# Language: Julia 1.7.2

`https://julialang.org/downloads/`



The screenshot shows the Julia 1.7.2 download page. The navigation bar includes links for Download, Documentation, Blog, Community, Learn, Research, JSOC, and a Sponsor button. The main heading is "Download Julia" with a GitHub star count of 39,047. A message encourages users to star the project on GitHub, cite it, or sponsor it. The current stable release is v1.7.2 (Feb 6, 2022). A table lists download links for various operating systems and architectures, including Windows, macOS (x86 and ARM), Generic Linux (x86 and ARM), Generic FreeBSD (x86), and Source code. Checksums are available in MD5 and SHA256 formats.

**Download Julia**

Star 39,047

Please star us on [GitHub](#). If you use Julia in your research, please [cite us](#). If possible, do consider [sponsoring us](#).

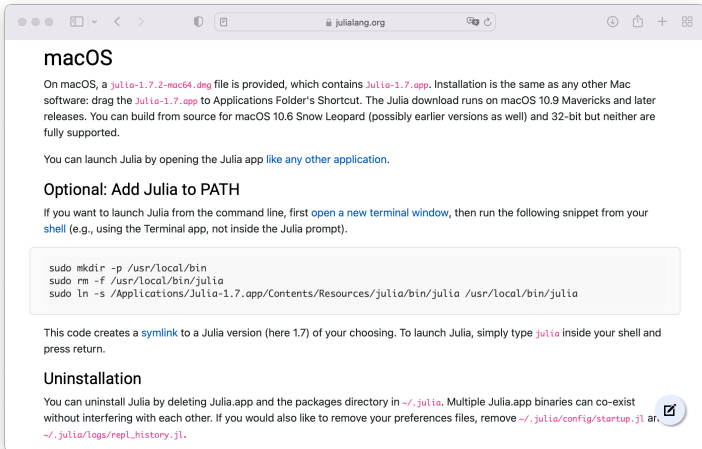
**Current stable release: v1.7.2 (Feb 6, 2022)**

Checksums for this release are available in both [MD5](#) and [SHA256](#) formats.

|                                                              |                                                          |                                       |
|--------------------------------------------------------------|----------------------------------------------------------|---------------------------------------|
| <b>Windows</b> <a href="#">[help]</a>                        | 64-bit (installer), 64-bit (portable)                    | 32-bit (installer), 32-bit (portable) |
| <b>macOS x86 (Intel or Rosetta)</b> <a href="#">[help]</a>   | 64-bit (.dmg), 64-bit (.tar.gz)                          |                                       |
| <b>macOS ARM (M-series Processor)</b> <a href="#">[help]</a> | 64-bit (experimental)                                    |                                       |
| <b>Generic Linux on x86</b> <a href="#">[help]</a>           | 64-bit (glibc) (GPG), 64-bit (musl) <sup>[1]</sup> (GPG) | 32-bit (GPG)                          |
| <b>Generic Linux on ARM</b> <a href="#">[help]</a>           | 64-bit (AArch64) (GPG)                                   | 32-bit (ARMv7-a hard float) (GPG)     |
| <b>Generic FreeBSD on x86</b> <a href="#">[help]</a>         | 64-bit (GPG)                                             |                                       |
| <b>Source</b>                                                | Tarball (GPG)      Tarball with dependencies (GPG)       | GitHub                                |

# Language: Julia 1.7.2

<https://julialang.org/downloads/>

A screenshot of a web browser window showing the Julia 1.7.2 macOS installation page. The browser's address bar shows 'julialang.org'. The page title is 'macOS'. The main text describes the installation process: downloading a 'julia-1.7.2-mac64.dmg' file, dragging the 'Julia-1.7.app' to the Applications folder, and building from source for older macOS versions. It also mentions launching Julia like any other application. A section titled 'Optional: Add Julia to PATH' provides a terminal snippet to create a symlink. The snippet is: 'sudo mkdir -p /usr/local/bin', 'sudo rm -f /usr/local/bin/julia', and 'sudo ln -s /Applications/Julia-1.7.app/Contents/Resources/julia/bin/julia /usr/local/bin/julia'. Below this, it states that this code creates a symlink to the chosen Julia version and that Julia can be launched by typing 'julia' in the shell. A final section titled 'Uninstallation' explains how to remove Julia by deleting the app and packages directory, and how to remove preference files. A small icon of a document with a pencil is visible in the bottom right corner of the page content area.

## macOS

On macOS, a [julia-1.7.2-mac64.dmg](#) file is provided, which contains [Julia-1.7.app](#). Installation is the same as any other Mac software: drag the [Julia-1.7.app](#) to Applications Folder's Shortcut. The Julia download runs on macOS 10.9 Mavericks and later releases. You can build from source for macOS 10.6 Snow Leopard (possibly earlier versions as well) and 32-bit but neither are fully supported.

You can launch Julia by opening the Julia app [like any other application](#).

### Optional: Add Julia to PATH

If you want to launch Julia from the command line, first [open a new terminal window](#), then run the following snippet from your [shell](#) (e.g., using the Terminal app, not inside the Julia prompt).

```
sudo mkdir -p /usr/local/bin
sudo rm -f /usr/local/bin/julia
sudo ln -s /Applications/Julia-1.7.app/Contents/Resources/julia/bin/julia /usr/local/bin/julia
```

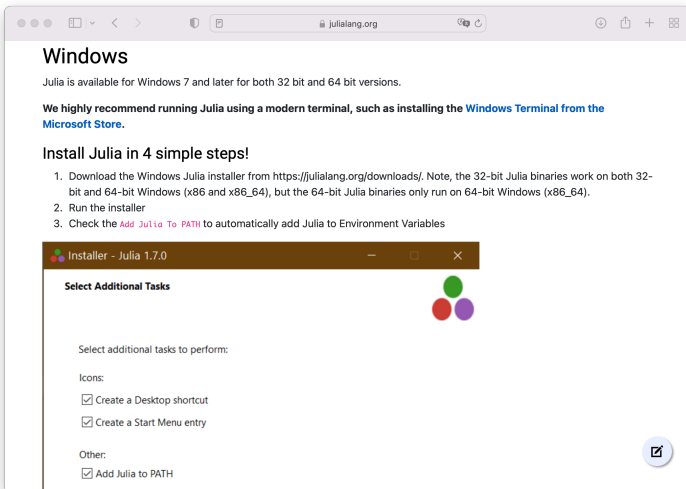
This code creates a [symlink](#) to a Julia version (here 1.7) of your choosing. To launch Julia, simply type [julia](#) inside your shell and press return.

### Uninstallation

You can uninstall Julia by deleting Julia.app and the packages directory in [~/julia](#). Multiple Julia.app binaries can co-exist without interfering with each other. If you would also like to remove your preferences files, remove [~/julia/config/startup.jl](#) or [~/julia/logs/repl\\_history.jl](#).

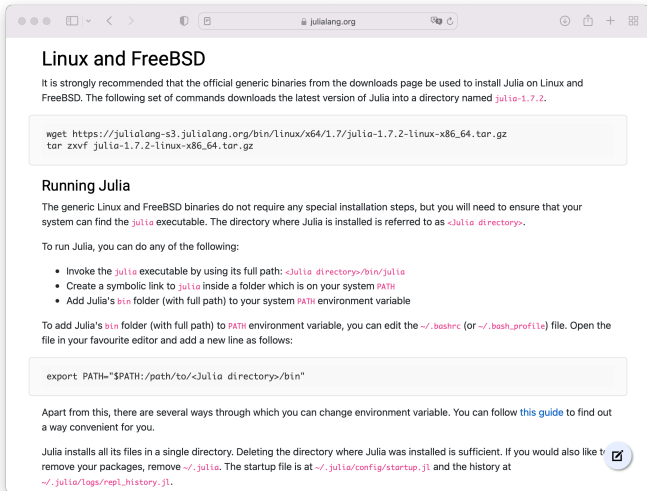
# Language: Julia 1.7.2

`https://julialang.org/downloads/`



# Language: Julia 1.7.2

<https://julialang.org/downloads/>



The screenshot shows a web browser window with the address bar displaying "julialang.org". The page title is "Linux and FreeBSD". The main content area contains the following text:

It is strongly recommended that the official generic binaries from the downloads page be used to install Julia on Linux and FreeBSD. The following set of commands downloads the latest version of Julia into a directory named `julia-1.7.2`.

```
wget https://julialang-s3.julialang.org/bin/linux/x64/1.7/julia-1.7.2-linux-x86_64.tar.gz
tar xzvf julia-1.7.2-linux-x86_64.tar.gz
```

**Running Julia**

The generic Linux and FreeBSD binaries do not require any special installation steps, but you will need to ensure that your system can find the `julia` executable. The directory where Julia is installed is referred to as `<Julia directory>`.

To run Julia, you can do any of the following:

- Invoke the `julia` executable by using its full path: `<Julia directory>/bin/julia`
- Create a symbolic link to `julia` inside a folder which is on your system `PATH`
- Add Julia's `bin` folder (with full path) to your system `PATH` environment variable

To add Julia's `bin` folder (with full path) to `PATH` environment variable, you can edit the `~/.bashrc` (or `~/.bash_profile`) file. Open the file in your favourite editor and add a new line as follows:

```
export PATH="$PATH:/path/to/<Julia directory>/bin"
```

Apart from this, there are several ways through which you can change environment variable. You can follow [this guide](#) to find out a way convenient for you.

Julia installs all its files in a single directory. Deleting the directory where Julia was installed is sufficient. If you would also like to remove your packages, remove `~/.julia`. The startup file is at `~/.julia/config/startup.jl` and the history at `~/.julia/logs/repl_history.jl`.

Language: Julia 1.7.2

Just click on the app icon



```
xgandibleux — julia — 80x24
```

```
Last login: Tue May 3 11:21:30 on ttys000
```

The default interactive shell is now zsh.

To update your account to use zsh, please run `chsh -s /bin/zsh`.

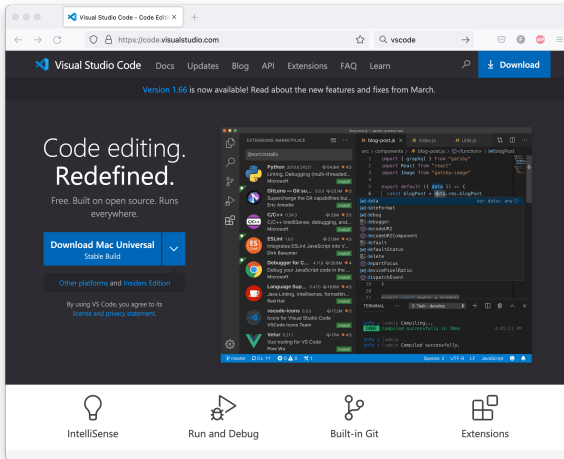
For more details, please visit <https://support.apple.com/kb/HT200050>.

```
MacBook-Pro-de-Xavier:~ xgandibleux$ exec '/Applications/Julia-1.7.app/Contents/Resources/julia/bin/julia'
```

```
(_) | Documentation: https://docs.julialang.org  
|_|  
||_| Type "?" for help, "]"? for Pkg help.  
||_| Version 1.7.2 (2022-02-06)  
||_| Official https://julialang.org/ release  
|_/_/  
  
julia> 
```

# Editor: Visual Studio Code

<https://code.visualstudio.com/>

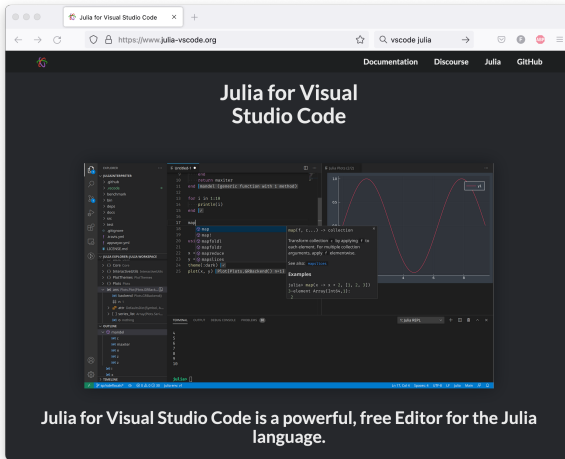


extended for Julia

<https://www.julia-vscode.org/>

# Editor: Visual Studio Code

<https://code.visualstudio.com/>



extended for Julia

<https://www.julia-vscode.org/>

# Editor: Visual Studio Code

<https://code.visualstudio.com/>

EXTENSIONS: PLACE ... ... Extension : Julia X

Search: julia

**Julia** v1.6.17  
julialang | 336 750 | ★★★★★ (70)  
Julia Language Support  
Désactiver | Désinstaller | ⚙️  
Cette extension est activée globalement.

Détails Contributions Journal des modifications État du runtime

## Julia

docs latest

This VS Code extension provides support for the Julia programming language.

### Getting started

Installing Julia/VS Code/VS Code Julia extension

1. Install Julia for your platform: <https://julialang.org/downloads/>
2. Install VS Code for your platform: <https://code.visualstudio.com/download> At the end of this step you should be able to start VS Code.
3. Choose **Install** in the VS Code Marketplace; or manually install with:
  1. Start VS Code.
  2. Inside VS Code, go to the extensions view either by executing the View: Show Extensions command (click View->Command Palette...) or by clicking on the extension icon on the left side of the VS Code window.
  3. In the extensions view, simply search for the term **julia** in the marketplace search box, then select the extension named **Julia** and click the install button. You might have to restart VS Code after this step.

### Configure the Julia extension

If you have installed Julia into a standard location on Mac or Windows, or if the Julia binary is on your PATH, the Julia VS Code extension should automatically find your Julia installation.

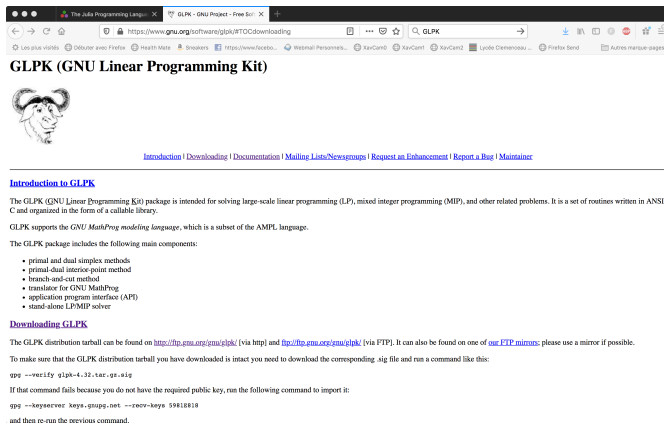
**Catégories**  
Programming Languages  
Snippets Linters  
Debuggers  
Data Science  
Notebooks  
Machine Learning

**Ressources**  
Place de marché  
Dépôt  
Licence  
[julia-vscode.org](https://code.visualstudio.com)

**Informations sur la Place de marché**  
Publié le 21/06/2016, 23:56:01  
Dernière mise à jour 07/04/2022, 19:16:44  
Identificateur: julia.lang.Language: julia

# Open Source MILP solver: GLPK

<https://www.gnu.org/software/glpk/>



**GLPK (GNU Linear Programming Kit)**



[Introduction](#) | [Downloading](#) | [Documentation](#) | [Mailing Lists/Newsletters](#) | [Request an Enhancement](#) | [Report a Bug](#) | [Maintainer](#)

---

### Introduction to GLPK

The GLPK (GNU Linear Programming Kit) package is intended for solving large-scale linear programming (LP), mixed integer programming (MIP), and other related problems. It is a set of routines written in ANSI C and organized in the form of a callable library.

GLPK supports the *GNU MathProg modeling language*, which is a subset of the AMPL language.

The GLPK package includes the following main components:

- primal and dual simplex methods
- primal-dual interior-point method
- branch-and-cut method
- translator for GNU MathProg
- application program interface (API)
- stand-alone LP/MIP solver

### Downloading GLPK

The GLPK distribution tarball can be found on <http://ftp.gnu.org/gnu/glpk/> (via http) and <ftp://ftp.gnu.org/gnu/glpk/> (via FTP). It can also be found on one of [our FTP mirrors](#); please use a mirror if possible.

To make sure that the GLPK distribution tarball you have downloaded is intact you need to download the corresponding .sig file and run a command like this:

```
gpg --verify glpk-4.32.tar.gz.sig
```

If that command fails because you do not have the required public key, run the following command to import it:

```
gpg --keyserver keys.gnupg.net --recv-keys 5981B818
```

and then re-run the previous command.

# Installing GLPK

## On macOS

The easy way to install GLPK with one of these two package manager:

- ▶ homebrew
  - ▶ install homebrew: <https://brew.sh/>
  - ▶ install GLPK: <https://formulae.brew.sh/formula/glpk>
- ▶ macport
  - ▶ install macport: <https://www.macports.org/>
  - ▶ install GLPK: <https://ports.macports.org/port/glpk/>

## On windows

Follow the instructions given here:

- ▶ <http://winglpk.sourceforge.net/>

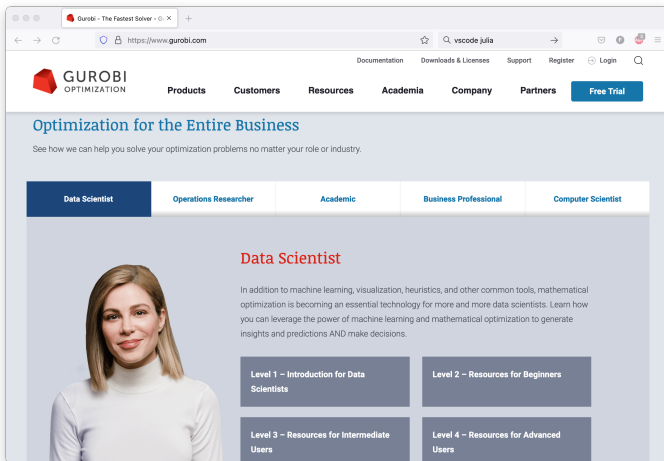
## On linux (ubuntu)

Follow the instructions given here:

- ▶ <https://howtoinstall.co/en/glpk>

# Commercial MILP solver: Gurobi

<https://www.gurobi.com/>



# Installing Gurobi (academic license)

Gurobi software downloads and license center:

<https://www.gurobi.com/downloads/>

Video for

- ▶ macOS:

<https://www.youtube.com/watch?v=ZcL-NmckTxQ>

- ▶ windows:

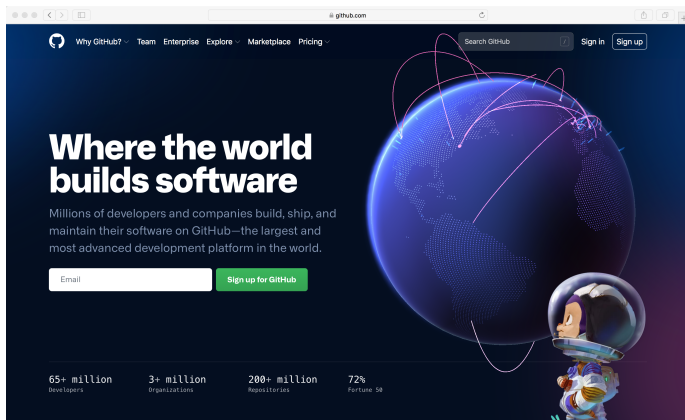
<https://www.youtube.com/watch?v=fQVxuW0iPpI&t=0s>

- ▶ linux (ubuntu):

<https://www.youtube.com/watch?v=yNmeG6Wom1o&t=0s>

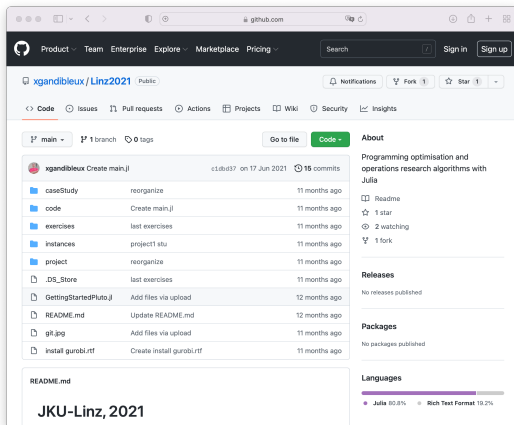
# Repository: GitHub and the app GitHub Desktop

`https://github.com` and `https://desktop.github.com`



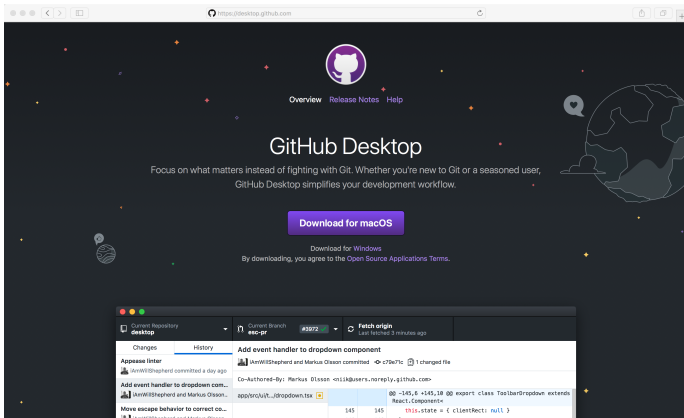
# Repository: GitHub and the app GitHub Desktop

`https://github.com` and `https://desktop.github.com`



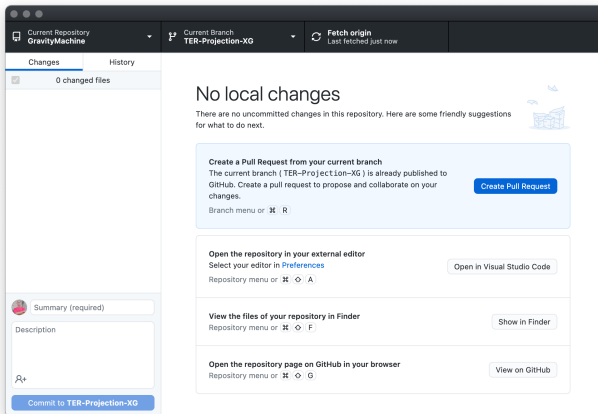
# Repository: GitHub and the app GitHub Desktop

<https://github.com> and <https://desktop.github.com>



# Repository: GitHub and the app GitHub Desktop

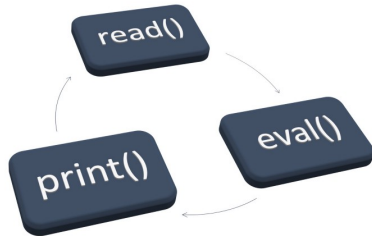
`https://github.com` and `https://desktop.github.com`





# Getting started

## RRead EEvaluate PPrint LLoop



## REPL: normal mode

Just click on the app icon



# REPL: normal mode (commands)

Some commands to know:

Load (and run) a program

`include("filename.jl")`

Interrupt execution

`[Ctrl] + [C]`

Recall last result

`ans`

Previous-run command

Up key

Search backwards into history

`[Ctrl] + [r]`

Tab-completion

`characters + [Tab]`

Clear screen

`[Ctrl] + [L]`

Exit REPL

`exit()` or `[Ctrl] + [D]`

# REPL: normal mode (commands)

Some commands to know:

|                               |                                                  |
|-------------------------------|--------------------------------------------------|
| Load (and run) a program      | <code>include("filename.jl")</code>              |
| Interrupt execution           | <code>[Ctrl] + [C]</code>                        |
| Recall last result            | <code>ans</code>                                 |
| Previous-run command          | Up key                                           |
| Search backwards into history | <code>[Ctrl] + [r]</code>                        |
| Tab-completion                | <code>characters + [Tab]</code>                  |
| Clear screen                  | <code>[Ctrl] + [L]</code>                        |
| Exit REPL                     | <code>exit()</code> or <code>[Ctrl] + [D]</code> |

# REPL: normal mode (commands)

Some commands to know:

|                               |                                                  |
|-------------------------------|--------------------------------------------------|
| Load (and run) a program      | <code>include("filename.jl")</code>              |
| Interrupt execution           | <code>[Ctrl] + [C]</code>                        |
| Recall last result            | <code>ans</code>                                 |
| Previous-run command          | Up key                                           |
| Search backwards into history | <code>[Ctrl] + [r]</code>                        |
| Tab-completion                | <code>characters + [Tab]</code>                  |
| Clear screen                  | <code>[Ctrl] + [L]</code>                        |
| Exit REPL                     | <code>exit()</code> or <code>[Ctrl] + [D]</code> |

# REPL: normal mode (commands)

Some commands to know:

|                               |                                                  |
|-------------------------------|--------------------------------------------------|
| Load (and run) a program      | <code>include("filename.jl")</code>              |
| Interrupt execution           | <code>[Ctrl] + [C]</code>                        |
| Recall last result            | <code>ans</code>                                 |
| Previous-run command          | Up key                                           |
| Search backwards into history | <code>[Ctrl] + [r]</code>                        |
| Tab-completion                | <code>characters + [Tab]</code>                  |
| Clear screen                  | <code>[Ctrl] + [L]</code>                        |
| Exit REPL                     | <code>exit()</code> or <code>[Ctrl] + [D]</code> |

## REPL: help mode

```
julia> ?
```

```
The default interactive shell is now zsh.  
To update your account to use zsh, please run `chsh -s /bin/zsh`.  
For more details, please visit https://support.apple.com/kb/HT208050.  
MacBook-Pro-de-Xavier:~ xgandibleux$ exec '/Applications/Julia-1.7.app/Contents/  
Resources/julia/bin/julia'  
  
Documentation: https://docs.julialang.org  
Type "?" for help, "]?" for Pkg help.  
Version 1.7.2 (2022-02-06)  
Official https://julialang.org/ release  
  
[help?> VERSION  
search: VERSION VersionNumber versioninfo reverseind  
  
VERSION  
  
A VersionNumber object describing which version of Julia is in use. For  
details see Version Number Literals.  
  
julia>
```

## REPL: shell mode

```
julia> ;
```

```

Last login: Wed May  4 14:07:17 on ttys003

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBook-Pro-de-Xavier:~ xgandibleux$ exec '/Applications/Julia-1.7.app/Contents/Resources/julia/bin/julia'

      _   _          _     _             | Documentation: https://docs.julialang.org
    _(_)_( )_        (_)   (_)           |
   (_)(_)(_)_       _/_/_/              | Type "?" for help, "]"? for Pkg help.
  _/___/_/___/_/_/_/_/_/_/_/_/_/_/_/_ |
 _/___/_/___/_/_/_/_/_/_/_/_/_/_/_/__| Version 1.7.2 (2022-02-06)
|__/_/                                   | Official https://julialang.org/ release

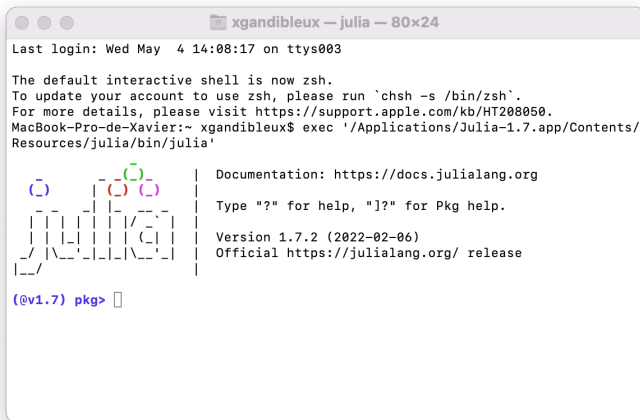
[shell> pwd
/Users/xgandibleux

shell> 
```

Return to normal mode: [Backspace] on empty line

# REPL: package manager mode

julia> ]



```
xgandibleux — julia — 80x24
Last login: Wed May  4 14:08:17 on ttys003

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBook-Pro-de-Xavier:~ xgandibleux$ exec '/Applications/Julia-1.7.app/Contents/
Resources/julia/bin/julia'

Documentation: https://docs.julialang.org
Type "?" for help, "]?" for Pkg help.
Version 1.7.2 (2022-02-06)
Official https://julialang.org/ release

(@v1.7) pkg> ]
```

Return to normal mode: [Backspace] on empty line

# REPL: package manager mode (commands)

Some commands to know:

|                                |              |
|--------------------------------|--------------|
| add package(s) to project      | add          |
| remove package(s) from project | remove or rm |
| summarize contents             | status or st |
| update packages                | update or up |
| help online                    | help         |



Getting started

# Packages

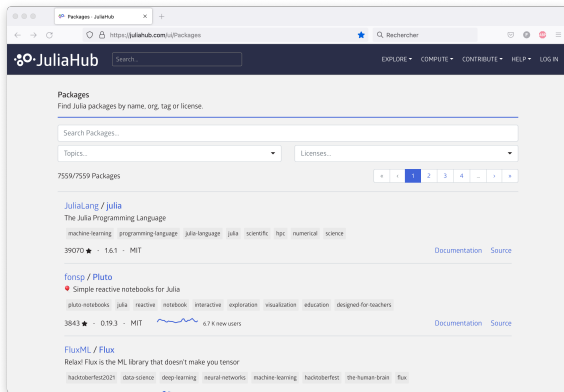


# Packages

Visual interface for exploring the packages:

Julia hub

<https://juliahub.com/ui/Packages>



Alternative:

Julia Packages

<https://juliapackages.com/>

# Packages

A selection of useful packages:

Standard library:

|                            |                                                   |
|----------------------------|---------------------------------------------------|
| <code>Pkg</code>           | primitives for managing packages                  |
| <code>Random</code>        | primitives for random number                      |
| <code>LinearAlgebra</code> | primitives for linear algebra                     |
| <code>Printf</code>        | primitives for C-like format to display on screen |

External library:

|                           |                                                              |
|---------------------------|--------------------------------------------------------------|
| <code>PyPlot</code>       | Plotting for Julia based on matplotlib                       |
| <code>Plots</code>        | a high-level plotting package                                |
| <code>JuMP</code>         | Modeling language for Mathematical Optimization              |
| <code>GLPK</code>         | interface for GLPK solver                                    |
| <code>Gurobi</code>       | interface for Gurobi solver                                  |
| <code>CPLEX</code>        | interface for the CPLEX solver                               |
| <code>vOptGeneric</code>  | Multiobjective linear optimization solver; generic problems  |
| <code>vOptSpecific</code> | Multiobjective linear optimization solver; specific problems |

# Packages

A selection of useful packages:

Standard library:

|               |                                                   |
|---------------|---------------------------------------------------|
| Pkg           | primitives for managing packages                  |
| Random        | primitives for random number                      |
| LinearAlgebra | primitives for linear algebra                     |
| Printf        | primitives for C-like format to display on screen |

External library:

|              |                                                              |
|--------------|--------------------------------------------------------------|
| PyPlot       | Plotting for Julia based on matplotlib                       |
| Plots        | a high-level plotting package                                |
| JuMP         | Modeling language for Mathematical Optimization              |
| GLPK         | interface for GLPK solver                                    |
| Gurobi       | interface for Gurobi solver                                  |
| CPLEX        | interface for the CPLEX solver                               |
| vOptGeneric  | Multiobjective linear optimization solver; generic problems  |
| vOptSpecific | Multiobjective linear optimization solver; specific problems |

# Packages

A selection of useful packages:

Standard library:

|                            |                                                   |
|----------------------------|---------------------------------------------------|
| <code>Pkg</code>           | primitives for managing packages                  |
| <code>Random</code>        | primitives for random number                      |
| <code>LinearAlgebra</code> | primitives for linear algebra                     |
| <code>Printf</code>        | primitives for C-like format to display on screen |

External library:

|                           |                                                              |
|---------------------------|--------------------------------------------------------------|
| <code>PyPlot</code>       | Plotting for Julia based on matplotlib                       |
| <code>Plots</code>        | a high-level plotting package                                |
| <code>JuMP</code>         | Modeling language for Mathematical Optimization              |
| <code>GLPK</code>         | interface for GLPK solver                                    |
| <code>Gurobi</code>       | interface for Gurobi solver                                  |
| <code>CPLEX</code>        | interface for the CPLEX solver                               |
| <code>vOptGeneric</code>  | Multiobjective linear optimization solver; generic problems  |
| <code>vOptSpecific</code> | Multiobjective linear optimization solver; specific problems |

# Adding a package (from the external library)

Commands to invoke **once** before the first use of the given package:

Method 1:

```
(@v1.7) pkg> add PyPlot  
(@v1.7) pkg> add JuMP, GLPK
```

Method 2:

```
julia> using Pkg  
julia> Pkg.add("PyPlot")  
julia> Pkg.add("JuMP")  
julia> Pkg.add("GLPK")
```

# Adding a package (from the external library)

Commands to invoke **once** before the first use of the given package:

Method 1:

```
(@v1.7) pkg> add PyPlot  
(@v1.7) pkg> add JuMP, GLPK
```

Method 2:

```
julia> using Pkg  
julia> Pkg.add("PyPlot")  
julia> Pkg.add("JuMP")  
julia> Pkg.add("GLPK")
```

# Using a package

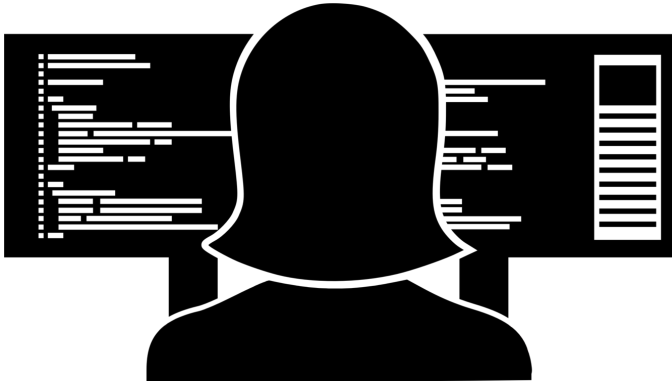
Commands to invoke **each time** before the first use of the given package:

```
julia> using Pkg  
julia> using JuMP, GLPK  
julia> using Printf  
julia> using LinearAlgebra
```



Getting started

Let's try



# Running instructions into the REPL

1) Click on the app icon...



... the REPL is ready!

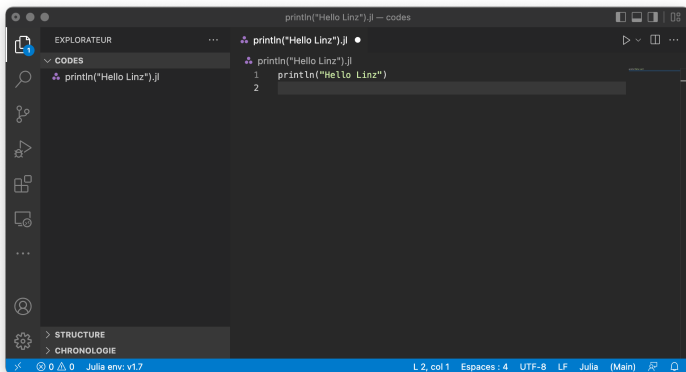
2) Type your instructions...

```
julia> println("Hello Linz")
```

... done!

# Running a program into the REPL

Program prepared with a text editor (here VScode)...



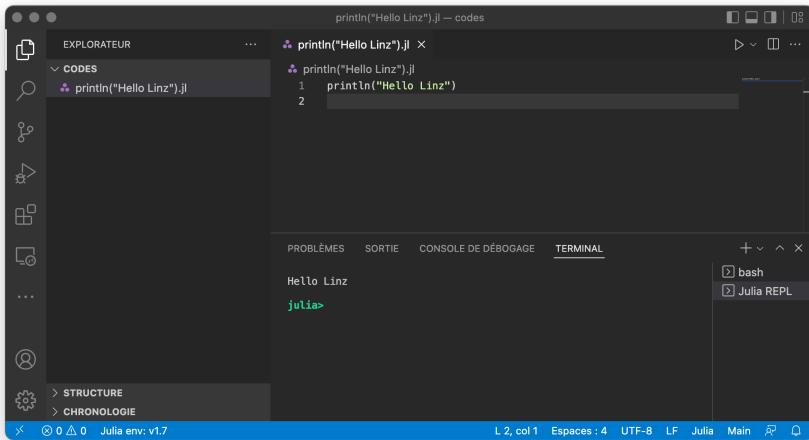
...to load and execute into the REPL :

```
julia> include("hello.jl")
```

# Running a program into an IDE (VScode)

Program prepared with a text editor (here VScode)...

...to load and execute into the VScode terminal:



# Running Julia code into a notebook (Jupyter)

**Jupyter** <https://jupyter.org/>

IJulia has to be installed

(<https://github.com/JuliaLang/IJulia.jl>)

Steps to follow:

```
(@v1.7) pkg> add IJulia
```

```
julia> using IJulia  
julia> notebook()
```

answer yes to the questions

Done!

# Running Julia code into a notebook (Jupyter)

**Jupyter** <https://jupyter.org/>

IJulia has to be installed

(<https://github.com/JuliaLang/IJulia.jl>)

Steps to follow:

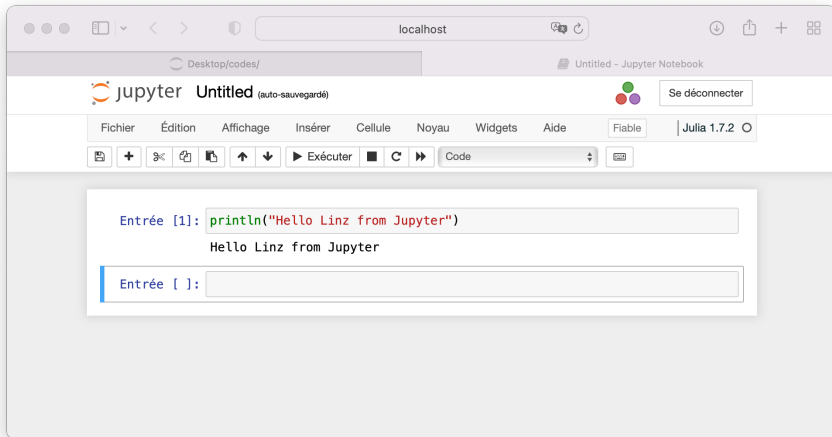
```
(@v1.7) pkg> add IJulia
```

```
julia> using IJulia  
julia> notebook()
```

answer yes to the questions

Done!

# Running Julia code into a notebook (Jupyter)



# Running Julia code into a notebook (Pluto)

**Pluto** <https://github.com/fonsp/Pluto.jl>

Install: <https://www.youtube.com/watch?v=C4QhZcX34mI>

Steps to follow:

```
(@v1.7) pkg> add Pluto
```

```
julia> using Pluto  
julia> Pluto.run()
```

Done!

# Running Julia code into a notebook (Pluto)

**Pluto** <https://github.com/fonsp/Pluto.jl>

Install: <https://www.youtube.com/watch?v=C4QhZcX34mI>

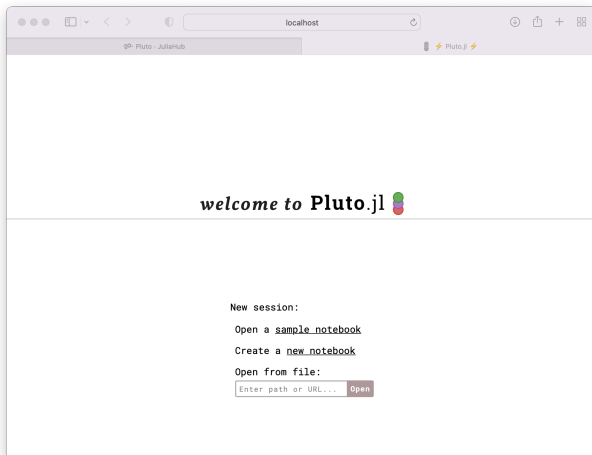
Steps to follow:

```
(@v1.7) pkg> add Pluto
```

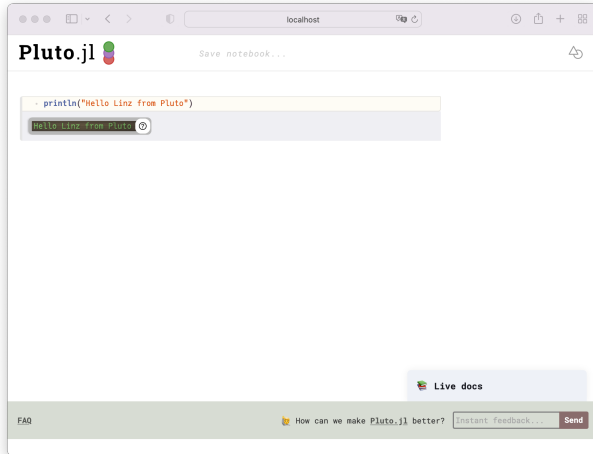
```
julia> using Pluto  
julia> Pluto.run()
```

Done!

# Running Julia code into a notebook (Pluto)



# Running Julia code into a notebook (2)





# Let's go for installing the apps and do your first runs

